

# MERN 16 || M-11 || Assignment

## Part 1: Theory Questions

Answer the following questions in your own words.

### Node.js Basics

**Q. What is [Node.js](#)?**

**Ans.** Node Js is an Environment which allows the JavaScript run out of Browser. We use it to develop server apps in the backend.

**Q. What is the difference between Single-threaded and Multi-threaded architecture?**

**Ans. Single-threaded:** Processes all tasks using a single sequence of execution (one thread). It handles tasks one by one or delegates heavy tasks to the system background, maximizing efficiency without using a lot of memory. Node.js uses this architecture.

**Multi-threaded:** Can spawn multiple threads to handle different tasks simultaneously. Each user request typically gets its own thread. While great for heavy calculations, it requires significantly more computer memory and resources.

**Q. What is the difference between Synchronous and Asynchronous operations?**

**Ans. Synchronous:** Tasks are executed sequentially, one after another. The program must completely finish the current task before moving on to the next line of code, meaning a slow task can "freeze" the app.

**Asynchronous:** Tasks can be started and sent to the background, allowing the program to immediately move to the next line of code. When the background task finishes, the program is notified and handles the result without any freezing.

**Q. What is the difference between Blocking and Non-blocking operations?**

**Ans. Blocking:** Refers to code execution that stops additional JavaScript from running until a non-JavaScript operation completes (like waiting for a file to read from a hard drive). It physically halts the thread.

**Non-blocking:** Allows the main JavaScript thread to remain free. Instead of waiting for a slow operation to finish, it registers a callback and moves on, keeping the application highly responsive.

## Modules

**Q. What is a Module in [Node.js](#)?**

**Ans.** A module is a reusable block of code isolated in its own file. Instead of putting thousands of lines of code into a single file, you break your application down into separate, manageable files (modules) that can be easily imported where needed.

**Q. What is the purpose of `module.exports`?**

**Ans.** `module.exports` is the mechanism Node.js uses to expose functions, objects, or variables from a file so that other files in the application can access and use them. It essentially controls what a module "gives away."

**Q. What is the purpose of `require()`?**

**Ans.** `require()` is a built-in function used to read and execute a JavaScript file, returning its exported contents. It is how you import core modules, local files, or third-party packages into my current file.

## NPM

**Q. What is NPM?**

**Ans.** NPM stands for **Node Package Manager**. It is the default package manager for Node.js and acts as an online registry hosting millions of open-source libraries. It also includes a command-line tool used to install, update, and manage those packages in our project.

**Q. What information is stored inside the `package.json` file?**

**Ans.** The `package.json` file is the manifest or "identity card" of our project. It stores:

- Metadata: Project name, version, description, author, and licensing.
- Dependencies: A list of third-party packages (and their versions) required for the project to run.
- Scripts: Custom command shortcuts (like `npm start` or `npm run dev`) to automate repetitive tasks.

## Core Modules

**Q. What is the purpose of the following Node.js modules?**

**Ans.** The purpose of Node.js modules is to break down a large application into smaller, reusable, and organized blocks of code. Instead of writing

thousands of lines of code in one massive file, modules allow you to isolate specific functionalities (like database connections, authentication, or utility functions) into separate files. This makes your code much easier to maintain, test, and share across different projects. Here is following modules purpose:

- ★ **HTTP Module:** It allows Node.js to transfer data over the network and build web servers. It can listen to network ports and handle incoming server requests and outgoing responses.
- ★ **URL Module:** It provides utilities for URL resolution and parsing. It splits a web address/URL string into easily readable components (like the host, pathname, and query parameters).
- ★ **File System (fs) Module:** It provides access to physical files on the computer. It allows you to read, write, update, create, and delete files and directories.
- ★ **Path Module:** It provides utilities for working with file and directory paths. It safely joins path segments together and ensures code works seamlessly across different operating systems (Windows, macOS, Linux) that use different path slash rules.
- ★ **Events Module:** It provides the `EventEmitter` class, which allows you to create, fire (emit), and listen for custom events within your code. It is the foundation of Node.js's entire event-driven architecture.